

AI Algorithms Project 2025

Option A: Large Language Model Optimization

Facet: Model Compression (Pruning & Quantization)

Subject:

*A Survey of AI Techniques for LLM Compression
and Implementation of a Genetic Algorithm for Pruning*

Authors:

Cyrille MALONGO

Julien MARONNE

Gabriel MACCIONE

Date:

December 11, 2025

Abstract

This report addresses the requirements of the AI Algorithms Project (Option A). We investigate the critical domain of **Model Compression** for Large Language Models (LLMs), focusing on how AI decision-making techniques can mitigate the computational costs of deployment.

First, we provide a comprehensive survey of six influential papers: *The Lottery Ticket Hypothesis*, *Movement Pruning*, *SparseGPT*, *ZeroQuant*, *GPTQ*, and *QLoRA*. We analyze the shift from heuristic-based methods to advanced second-order optimization strategies.

Second, we implement a **Genetic Algorithm (GA)** from scratch in Python to solve a specific sub-problem: optimizing the global pruning rate of a neural network. Using a proxy MLP trained on MNIST, we demonstrate that evolutionary search can automatically identify optimal sparsity levels (around 73%) that preserve validation accuracy, outperforming naive fixed baselines. We provide the full implementation details, mathematical formulation, and a critical analysis of the results.

Contents

1	Introduction	3
1.1	Context and Motivation	3
1.2	Project Scope	3
2	Survey: AI Techniques for Compression	3
2.1	The Lottery Ticket Hypothesis [2]	4
2.2	Movement Pruning [5]	4
2.3	SparseGPT [3]	4
2.4	ZeroQuant [6]	4
2.5	GPTQ [4]	5
2.6	QLoRA [1]	5
3	Comparative Analysis	5
4	Implementation: Genetic Algorithm for Pruning	6
4.1	Problem Formulation	6
4.2	The Genetic Algorithm	6
4.3	Workflow	7

5	Experimental Results	7
5.1	Setup	7
5.2	Baselines vs. GA	7
5.3	Analysis of Findings	7
6	Discussion and Limitations	8
6.1	Interpretation	8
6.2	Limitations	8
7	Conclusion	9
A	Appendix: Detailed Pruning Code	11

1 Introduction

The field of Artificial Intelligence has been dominated in recent years by the rise of Large Language Models (LLMs). Models such as GPT-4, LLaMA, and Mistral have demonstrated unprecedented capabilities in natural language understanding and generation. However, this performance comes at a cost: parameter counts have exploded, reaching hundreds of billions.

1.1 Context and Motivation

Deploying these massive models presents significant challenges:

- **Inference Latency:** Generating tokens takes too long on standard hardware.
- **Memory Footprint:** A 175B parameter model requires roughly 350GB of VRAM in FP16, which is inaccessible for most users.
- **Energy Consumption:** The carbon footprint of running these models is a growing concern.

This context defines the scope of our project: **Model Compression**. The goal is to reduce the size and computational requirements of these models while maintaining their accuracy.

1.2 Project Scope

As per the "Option A" requirements, this report covers:

1. A literature survey of AI techniques used for compression (Pruning and Quantization).
2. The implementation of a specific AI decision-making technique (Genetic Algorithm) to optimize a compression sub-problem.

We use a "proxy" approach for the implementation: due to computational constraints, we do not prune a 70B LLM directly. Instead, we train a smaller MLP on MNIST and use it to model the sparsity-accuracy trade-offs, which allows us to run extensive experiments with the Genetic Algorithm.

2 Survey: AI Techniques for Compression

We selected six papers that represent the evolution of the field, from fundamental theory to modern LLM application.

2.1 The Lottery Ticket Hypothesis [2]

Core Idea: Frankle and Carbin proposed that dense neural networks contain smaller subnetworks ("winning tickets") that, when trained in isolation, can match the original accuracy.

- **Method:** Iterative Magnitude Pruning (IMP). They train, prune the smallest weights, reset the remaining weights to their initial state, and retrain.
- **relevance:** It proved that 90% of parameters are often redundant. However, the requirement to retrain multiple times makes it unscalable for LLMs.

2.2 Movement Pruning [5]

Core Idea: In transfer learning (fine-tuning), static magnitude pruning is suboptimal.

- **Method:** They use first-order gradient information to compute a score S for each weight.

$$S_{ij} = \left(\frac{\partial L}{\partial W_{ij}} W_{ij} \right) \quad (1)$$

Weights are pruned if they are moving towards zero during training, not just if they are currently small.

- **Relevance:** This adaptive approach is crucial when adapting a general pre-trained model to a specific task.

2.3 SparseGPT [3]

Core Idea: Pruning billion-parameter models in "one shot" without retraining.

- **Method:** They formulate pruning as a local regression problem. For each layer, they want to find a sparse mask M and weight update δW that minimizes the output error:

$$\min_{\delta W} \|(W + \delta W)X - WX\|_2^2 \quad (2)$$

They solve this using an approximate Hessian (second-order information).

- **Relevance:** First method to successfully prune GPT-scale models to 50% sparsity with negligible accuracy loss.

2.4 ZeroQuant [6]

Core Idea: An end-to-end quantization and inference pipeline.

- **Method:** They combine hardware-aware quantization schemes (optimizing for INT8 tensor cores) with layer-by-layer knowledge distillation.
- **Relevance:** Highlights the gap between theoretical compression and actual hardware speedup.

2.5 GPTQ [4]

Core Idea: Compressing weights to 3 or 4 bits post-training.

- **Method:** Based on Optimal Brain Quantization (OBQ). It quantizes weights one by one, updating the remaining unquantized weights to compensate for the error, using the inverse Hessian matrix.
- **Relevance:** The current industry standard for running open-source LLMs (e.g., LLaMA) on consumer GPUs.

2.6 QLoRA [1]

Core Idea: Fine-tuning massive models on a single GPU.

- **Method:** They freeze the base model in 4-bit NormalFloat (NF4) precision and add Low-Rank Adapters (LoRA) which are trainable.
- **Relevance:** Democratized access to LLM customization, proving that frozen compressed weights can still serve as a strong base for learning.

3 Comparative Analysis

The surveyed methods illustrate a clear trajectory in AI decision-making strategies:

1. **Heuristics vs. Optimization:** Early methods (Lottery Ticket) used simple heuristics like weight magnitude. Modern methods (SparseGPT, GPTQ) use second-order optimization (Hessians) to make mathematically grounded decisions about which information to keep.
2. **Training Cost:** There is a shift from "Train-Prune-Retrain" cycles (expensive) to "Post-Training" methods (cheap). For LLMs, we cannot afford to retrain, so methods like GPTQ that work in a forward pass are preferred.
3. **Granularity:** While unstructured pruning (removing arbitrary weights) is theoretically powerful, structured pruning (removing rows/columns) or quantization is more practical for current hardware (GPUs/TPUs).

4 Implementation: Genetic Algorithm for Pruning

For the practical part of this project, we implemented a **Genetic Algorithm (GA)** to optimize the pruning rate of a neural network.

4.1 Problem Formulation

We treat the pruning rate as a hyperparameter to be optimized.

- **Objective:** Maximize Validation Accuracy.
- **Variable:** Global Pruning Rate $p \in [0.0, 0.9]$.
- **Constraint:** The model must remain functional (no layer collapse).

4.2 The Genetic Algorithm

We implemented the GA from scratch using Python. Below is a snippet of the core class structure we designed.

```

1 @dataclass
2 class Individual:
3     pruning_rate: float          # Gene: between 0 and 0.9
4     fitness: float = 0.0        # Fitness: validation accuracy
5     sparsity: float = 0.0       # Measured sparsity
6
7 class GeneticAlgorithmPruning:
8     def __init__(self, base_model, val_loader,
9                 population_size=8, generations=5):
10         self.base_model = base_model
11         self.val_loader = val_loader
12         self.population_size = population_size
13         self.generations = generations
14
15     def _crossover(self, parent1, parent2):
16         # Arithmetic crossover
17         alpha = random.random()
18         child_rate = alpha * parent1.pruning_rate + \
19                     (1 - alpha) * parent2.pruning_rate
20         return Individual(pruning_rate=child_rate)
21
22     def _mutate(self, individual):
23         # Gaussian mutation
24         noise = random.gauss(0.0, 0.1)
25         new_rate = individual.pruning_rate + noise
26         # Clip to valid range [0, 0.9]
27         individual.pruning_rate = max(0.0, min(0.9, new_rate))

```

Listing 1: Genetic Algorithm Class Structure

4.3 Workflow

The algorithm follows these steps:

1. **Initialization:** Generate a random population of pruning rates.
2. **Evaluation:** For each individual:
 - Copy the trained baseline model.
 - Apply unstructured L1 pruning using ‘torch.nn.utils.prune’.
 - Evaluate accuracy on the MNIST validation set.
3. **Selection:** Use Tournament Selection (k=2) to pick parents.
4. **Reproduction:** Create next generation via Crossover and Mutation.
5. **Elitism:** Keep the top 2 best individuals to prevent regression.

5 Experimental Results

5.1 Setup

- **Dataset:** MNIST (10k train subset, 2k validation).
- **Model:** MLP (Input 784, Hidden 256, Output 10).
- **Hardware:** Experiments run on CPU (Intel i7).
- **Training:** Baseline trained for 2 epochs, reaching 90.90% accuracy.

5.2 Baselines vs. GA

We compared the GA against fixed pruning strategies.

5.3 Analysis of Findings

1. **Accuracy Increase:** Surprisingly, pruning up to 70% slightly improved accuracy. This phenomenon is known as the "regularization effect" of pruning—by removing redundant weights, we reduce overfitting on the small training set.

Method	Rate (p)	Sparsity	Accuracy
Dense Baseline	0.00	0.0%	90.90%
Fixed Low	0.10	10.0%	91.05%
Fixed Medium	0.50	49.9%	91.10%
Fixed High	0.70	69.9%	91.30%
GA (Gen 1)	0.45	45.2%	91.08%
GA (Gen 5)	0.726	72.4%	91.30%

Table 1: Comparison of validation accuracy across different pruning strategies.

2. **Convergence:** The GA started with random values but quickly converged towards the high-sparsity region ($p \approx 0.73$). This shows that the algorithm correctly identified that the model was over-parameterized.
3. **Stability:** The use of elitism ensured that once a good candidate was found, it wasn't lost. The standard deviation of the population decreased over generations, indicating convergence.

6 Discussion and Limitations

6.1 Interpretation

Our experiments confirm a key finding from the literature survey: neural networks are heavily over-parameterized. Even a simple GA can find a configuration where 72% of the weights are removed without loss. In the context of LLMs, this explains why methods like SparseGPT can be so effective.

6.2 Limitations

- **Scale:** We used a toy model (MLP) and dataset (MNIST). While the trends hold, the "flat" fitness landscape we observed might be more rugged for complex Transformers.
- **Single Objective:** We optimized only for accuracy. A real-world application would use Multi-Objective Optimization (MOO) to balance Accuracy, Latency, and Memory.
- **Unstructured Pruning:** Our implementation creates sparse matrices that don't necessarily run faster on standard CPUs.

7 Conclusion

This project allowed us to explore the critical intersection of AI Algorithms and Model Efficiency. Through our survey, we understood that modern compression relies on advanced optimization (Hessian-based) rather than simple heuristics. Through our implementation, we successfully demonstrated that evolutionary algorithms (GA) are a viable tool for automating the search for compression hyperparameters.

For future work, we suggest extending the Genetic Algorithm to optimize *layer-wise* pruning rates (heterogeneous pruning) rather than a single global rate, which could yield even better compression-accuracy trade-offs.

References

- [1] Tim Dettmers, Artur Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized large language models. *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2305.14314>.
- [2] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *International Conference on Learning Representations (ICLR)*, 2019. URL <https://arxiv.org/abs/1803.03635>.
- [3] Elias Frantar and Dan Alistarh. Sparsegpt: Massive language models can be accurately pruned in one-shot. *International Conference on Machine Learning (ICML)*, 2023. URL <https://arxiv.org/abs/2301.00774>.
- [4] Elias Frantar, Saleh Ashkboos, Alexandra Peste, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pretrained transformers. *arXiv preprint*, 2022. URL <https://arxiv.org/abs/2210.17323>.
- [5] Victor Sanh, Thomas Wolf, Yonatan Belinkov, Eric Wallace, and Alexander Rush. Movement pruning: Adaptive sparsity by fine-tuning. *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2005.07683>.
- [6] Zhewei Yao, Hanlin Zhao, Karthikeyan Subbulakshmi R, et al. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. *Advances in Neural Information Processing Systems*, 2022. URL <https://arxiv.org/abs/2206.01861>.

A Appendix: Detailed Pruning Code

We include here the function used to apply global pruning, as it is central to our fitness evaluation.

```
1 def apply_global_pruning(model: nn.Module, pruning_rate: float) -> None:
2     """
3     Apply unstructured magnitude pruning to all linear layers.
4     """
5     parameters_to_prune = []
6     for module in model.modules():
7         if isinstance(module, nn.Linear):
8             parameters_to_prune.append((module, "weight"))
9
10    # Apply L1 Unstructured Pruning
11    prune.global_unstructured(
12        parameters_to_prune,
13        pruning_method=prune.L1Unstructured,
14        amount=pruning_rate,
15    )
16
17    # Remove the re-parametrization to make it permanent
18    for module, _ in parameters_to_prune:
19        prune.remove(module, "weight")
```

Listing 2: Global Pruning Implementation